🐼 Mastering pandas groupby, crosstab, and melt

📚 Introduction

- `pandas` is a powerful Python library for data analysis. Three of its most useful tools for summarizing and reshaping data are `groupby`, `crosstab`, and `melt`. Understanding how to use these functions effectively can greatly enhance your data analysis capabilities.

🔷 groupby: Grouping and Aggregating Data

🔧 Syntax

```
DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=Tru
```

🧪 Example: Sales Data

Let's create a synthetic dataset representing sales transactions:

```
import pandas as pd

# Sample sales data
data = {
    'Region': ['North', 'South', 'East', 'West', 'North', 'South', 'East', 'West'],
    'Product': ['A', 'A', 'B', 'B', 'A', 'B', 'A', 'B'],
    'Sales': [250, 150, 200, 300, 400, 120, 330, 500]
}

df = pd.DataFrame(data)
print(df)
```

Output: 🆗

Region	Product	Sales
North	A	250
South	A	150

Region	Product	Sales
East	B	200
West	B	300
North	A	400
South	B	120
East	A	330
West	B	500

📊 Grouping by Region

```
# Total sales by region
region_sales = df.groupby('Region')['Sales'].sum()
print(region_sales)
```

Output:

Region	Sales
East	530
North	650
South	270
West	800
Name: Sales, dtype: int64	

📈 Grouping by Region and Product

```
# Total sales by region and product
region_product_sales = df.groupby(['Region', 'Product'])['Sales'].sum()
print(region_product_sales)
```

Output: 🆖

Region	Product	Sales
East	A	330
East	B	200
North	A	650
South	A	150
South	B	120
West	B	800

📌 Common Aggregation Functions

- `.sum()` – Total sum
- `.mean()` – Average
- `.count()` – Count of non-NA/null entries
- `.min()` / `.max()` – Minimum / Maximum
- `.agg()` – Multiple aggregations 🆗 🆗

```
# Multiple aggregations
aggregated = df.groupby('Product')['Sales'].agg(['sum', 'mean', 'count'])
print(aggregated)
```

Output:

Product	Sum	Mean	Count
A	830	276.666667	3
B	1120	280.000000	4

🔷 crosstab: Cross-Tabulation of Categorical Data

🔧 Syntax

```
pd.crosstab(index, columns, values=None, rownames=None, colnames=None, aggfunc=None, ma
```

🧪 Example: Customer Preferences

- Let's create a synthetic dataset representing customer preferences:

```
# Sample customer data
data = {
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female', 'Male', 'Male', 'Female'],
    'Preference': ['Tea', 'Coffee', 'Tea', 'Coffee', 'Tea', 'Tea', 'Coffee', 'Coffee']
}

df = pd.DataFrame(data)
print(df)
```

Output: 🆗

Gender	Preference
Male	Tea
Female	Coffee
Female	Tea
Male	Coffee
Female	Tea
Male	Tea
Male	Coffee
Female	Coffee

📊 Basic Crosstab

```
# Frequency table of Gender vs Preference
ct = pd.crosstab(df['Gender'], df['Preference'])
print(ct)
```

Output: 🆗

Gender	Coffee	Tea
Female	2	2

Gender	Coffee	Tea
Male	2	2

📈 Crosstab with Aggregation

- Suppose we have scores associated with each entry:

```
# Adding scores to the dataset
df['Score'] = [85, 90, 88, 75, 95, 80, 70, 92]
print(df)
```

Output:

Gender	Preference	Score
Male	Tea	85
Female	Coffee	90
Female	Tea	88
Male	Coffee	75
Female	Tea	95
Male	Tea	80
Male	Coffee	70
Female	Coffee	92

```
# Average score by Gender and Preference
ct_avg = pd.crosstab(df['Gender'], df['Preference'], values=df['Score'], aggfunc='mean'
print(ct_avg)
```

Output:

Gender	Coffee	Tea
Female	91.0	91.5
Male	72.5	82.5

📌 Adding Margins and Normalization

```
# Crosstab with totals and normalized values
ct_norm = pd.crosstab(df['Gender'], df['Preference'], margins=True, normalize='index')
print(ct_norm)
```

Output:

Gender	Coffee	Tea	All
Female	0.5	0.5	1.0
Male	0.5	0.5	1.0
All	0.5	0.5	1.0

🔷 melt: Reshaping Data from Wide to Long Format

🔧 Syntax

```
pd.melt(frame, id_vars=None, value_vars=None, var_name=None, value_name='value', col_le
```

🧪 Example: Student Scores

- Let's create a synthetic dataset representing student scores:

```
# Sample student data
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Math': [85, 90, 78],
    'Science': [88, 92, 84],
    'History': [82, 85, 80]
}

df = pd.DataFrame(data)
print(df)
```

Output:

Name	Math	Science	History
Alice	85	88	82
Bob	90	92	85
Charlie	78	84	80

📊 Melting the DataFrame

```
# Reshaping from wide to long format
melted_df = pd.melt(df, id_vars=['Name'], var_name='Subject', value_name='Score')
print(melted_df)
```

Output: 🆗

Name	Subject	Score
Alice	Math	85
Bob	Math	90
Charlie	Math	78
Alice	Science	88
Bob	Science	92
Charlie	Science	84
Alice	History	82
Bob	History	85
Charlie	History	80

📌 Customizing Column Names

```
# Customizing the variable and value column names
melted_df = pd.melt(df, id_vars=['Name'], var_name='Subject', value_name='Score')
print(melted_df)
```

Output:

Name	Subject	Score
Alice	Math	85
Bob	Math	90
Charlie	Math	78
Alice	Science	88
Bob	Science	92
Charlie	Science	84